

No linear

UNIT 2: HASHING OR DISPERSED FUNCTIONS

Hashing Fundamentals, Hash Functions, Collision Resolution

Author: Eng. Carlos Andrés Sierra, M.Sc.
cavirguezs@udistrital.edu.co

Full-time Adjunct Professor
Computer Engineering Program
School of Engineering
Universidad Distrital Francisco José de Caldas

2026-II

Graph



UNIVERSIDAD DISTRITAL
FRANCISCO JOSÉ DE CALDAS

Course Outline

1 Hashing Fundamentals

2 Hash Functions

3 Collisions

4 Analysis



Outline

- 1 Hashing Fundamentals
- 2 Hash Functions
- 3 Collisions
- 4 Analysis



Concept of Hashing (Dispersion) I

Hashing

A mapping technique that converts an arbitrary search key into a valid integer index within a fixed-size array mapping target, enabling near $O(1)$ constant time execution access.

Direct Addressing vs. Hash Tables

- Direct Addressing:** Ideal case where every possible key maps uniquely to an exact memory slot. Requires space proportional to the universe of keys ($|U|$).
- Hash Table:** Maps massive key domains to a compressed table capacity (m), where $m \ll |U|$.

data representation



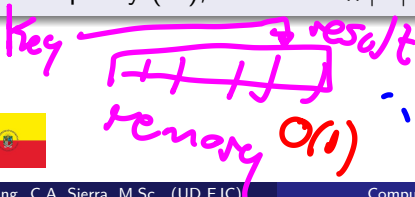
Concept of Hashing (Dispersion) I

Hashing

A mapping technique that converts an arbitrary search key into a valid integer index within a fixed-size array mapping target, enabling *near $O(1)$ constant time execution access*.

Direct Addressing vs. Hash Tables

- **Direct Addressing:** Ideal case where every possible key maps uniquely to an exact memory slot. Requires space proportional to the universe of keys (U).
- **Hash Table:** Maps massive key domains to a compressed table capacity (m), where $m \ll |U|$.



Concept of Hashing (Dispersion) II

Load Factor (α)

The operational density metric of a table capacity size defined as:

$$\alpha = \frac{n}{m} = \left(\frac{\text{Number of stored keys}}{\text{Total table size}} \right)$$

As $\alpha \rightarrow 1$ (or higher in chaining), the probability of index mapping collisions escalates exponentially.

10 students - 4 stored keys $\Rightarrow \alpha = 0.4$

key field Keys table size Field

f(x)

1	→	C-D-M-J-S	→	5	✓	~ 74%
2	→	S-J-M-M-J	→	5	✓	~ 74%
3	→	A-E-C-K-J-G	→	6	(+1)	~ 68%
4	→	B-E-A	→	3	(-2)	80%



Outline

- 1 Hashing Fundamentals
- 2 Hash Functions**
- 3 Collisions
- 4 Analysis



Classical Hash Functions I

1. Modulo Method (Division)

Uses the algebraic remainder of integer division.

$$h(\text{key}) = \text{key} \pmod{m}$$

Design rule: Table size m should be a prime number not close to powers of 2 to distribute indexes evenly.

mod 2 $\rightarrow$ 0-1

mod 10 $\rightarrow$ 0-1-2-...-9

mod 5

m slots

0-5 M M J A A
 1-K
 2-3 C C B
 3-C M J
 4-A J J E N

0-5 J
 1-
 2-5 M C E
 3-n
 4-J G W
 5-A

6-x
 7-M B
 8-C A U
 9-A J



Classical Hash Functions II

2. Mid-Square Method

Squares the key (k^2) and extracts the specified central digits.

- Example:

key = 45 $\Rightarrow 45^2 = 2025 \Rightarrow$ Extract middle digits $\rightarrow 02$.

Handwritten notes and calculations illustrating the Mid-Square Method:

- $46^2 = 2116$ (written in pink)
- $44^2 = 1936$ (written in pink)
- A long division calculation: $1999 \overline{) 2025015}$ with a remainder of 11 (written in blue).
- A 3x3 grid of numbers: $\begin{matrix} 9 & 7 & 2 & 4 & 8 \\ 1 & 9 & 3 & 6 \\ 2 & 0 & 2 & 5 \\ 2 & 1 & 1 & 6 \end{matrix}$ (written in red, blue, and pink). A blue bracket groups the top row, and a blue arrow points from the bottom row to the word "load".
- The word "load" is written in blue.
- The word "Factor" is written in blue.
- The word "key field" is written in blue.
- The function $f(x)$ is written in blue.
- The word "sensible" is written in pink.
- The word "range" is written in blue.



Classical Hash Functions III

3. Truncation Method

Extracts and concatenates arbitrary digits from pre-selected key positions while discarding others.



20261020112 $\Rightarrow$ $f(x)$
62

28	38 +
19	36
53	34
47	37
	25

collisions $\rightarrow$ 30
22 + + + +
32



Classical Hash Functions IV

4. Base Conversion

Interprets the original key in base p , then converts it to base q to isolate stable indexes.

DEC - HEX $\rightarrow$ break linearity

15	-	F
16	-	10

17 - 11

ID $\rightarrow$ Turn $n-1$ $\rightarrow$ HEX

crypto $\rightarrow$ metric space



Outline

1 Hashing Fundamentals

2 Hash Functions

3 Collisions

4 Analysis

$$a \cdot b = b \cdot a$$

$$b + a = a + b$$

ca@w.edu.co
ac@w.edu.co } same ASCII

Hashing Ideal

Full collision

$$O(n)$$

$$O(n^2)$$

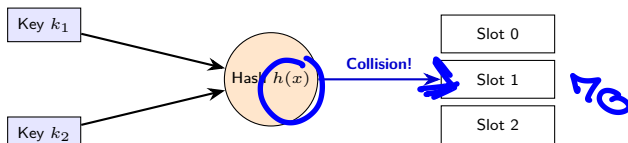


The Concept of a Collision

Definition

Occurs when two or more distinct keys generate the exact same hash index output.

$$h(k_1) = h(k_2) \quad \text{where } k_1 \neq k_2$$



Because $|U| > m$, by the pigeonhole principle, avoiding collisions completely is impossible. Structural resolution systems are required.

Handwritten examples:

- 4047 1055 111 → 01
- 2022 1020471 → 01

no relation

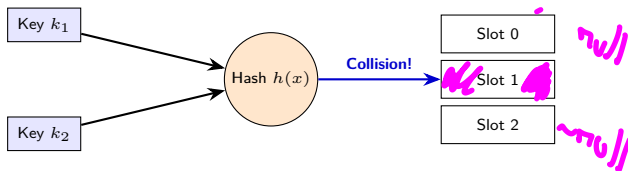


The Concept of a Collision

Definition

Occurs when two or more distinct keys generate the exact same hash index output.

$$h(k_1) = h(k_2) \quad \text{where } k_1 \neq k_2$$



Because $|U| > m$ by the Pigeonhole Principle, avoiding collisions completely is impossible. Structural resolution systems are required.

$\uparrow$ pigeons $\rightarrow$ m holes, $\uparrow > m$
at least 1 pigeon



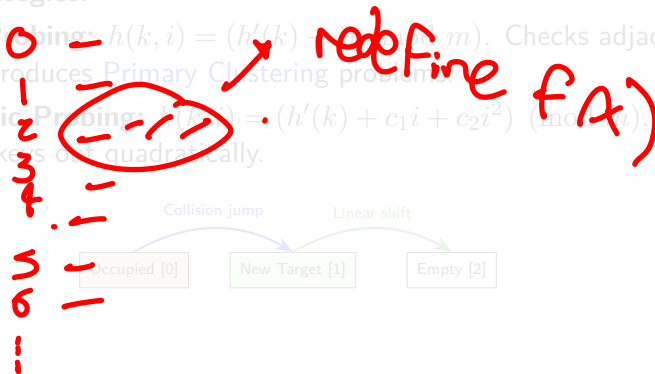
Collision Handling: *Open Addressing*

Reassignment Theory

All elements are stored directly inside the array table. If a collision occurs, we probe alternative slots sequentially until an empty cell is found.

Probing Strategies:

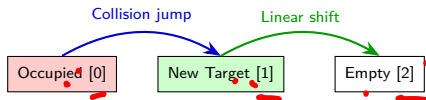
- **Linear Probing:** $h(k, i) = (h'(k) + i) \bmod m$. Checks adjacent slots. Introduces **Primary Clustering** problem.
- **Quadratic Probing:** $h(k, i) = (h'(k) + c_1 i + c_2 i^2) \bmod m$. Spreads keys out quadratically.



All elements are stored directly inside the array table. If a collision occurs, we probe alternative slots sequentially until an empty cell is found.

Probing Strategies: *sondeo i-j i+1*

- **Linear Probing:** $h(k, i) = (h'(k) + i) \pmod{m}$. Checks adjacent slots. Introduces Primary Clustering problems.
- **Quadratic Probing:** $h(k, i) = (h'(k) + c_1 i + c_2 i^2) \pmod{m}$. Spreads keys out quadratically.



Collision Handling: *Chaining & Buckets*

Nested Arrays (Buckets)

Each slot in the main table points to an array profile slot that can hold a fixed number of multiple items.

$f(x)$



Sequential Chaining

Each slot in the main table functions as a head pointer to a Singly Linked List of collided records.

Key A

Key B

0



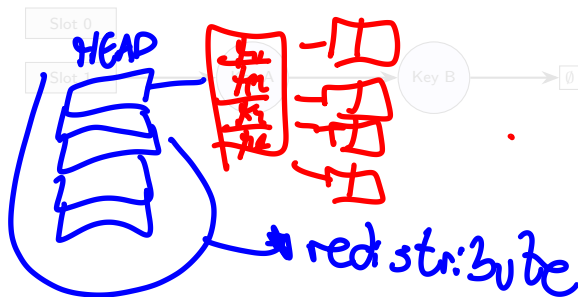
Collision Handling: *Chaining & Buckets*

Nested Arrays (Buckets)

Each slot in the main table points to an array profile slot that can hold a fixed number of multiple items.

Sequential Chaining

Each slot in the main table functions as a head pointer to a Singly Linked List of collided records.



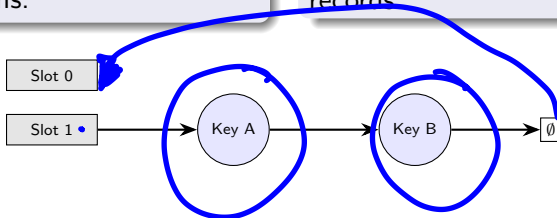
Collision Handling: *Chaining & Buckets*

Nested Arrays (Buckets)

Each slot in the main table points to an array profile slot that can hold a fixed number of multiple items.

Sequential Chaining

Each slot in the main table functions as a head pointer to a **Singly Linked List** of collided records



Outline

- 1 Hashing Fundamentals
- 2 Hash Functions
- 3 Collisions
- 4 Analysis



Performance Analysis

Access Complexity

- **Average Case:** $O(1)$ lookup operations, assuming an even, uniform distribution of key entries.
- **Worst Case:** $O(n)$ access operations if all inserted keys collide into the exact same index link sequence.

Search Strategies Trade-offs

Metric Type	Sequential	Binary Search	Hashing
Average Search Time	$O(n)$	$O(\log n)$	$O(1)$
Worst Search Time	$O(n)$	$O(\log n)$	$O(n)$
Memory Overhead	Minimum	Balanced	Table Space (α)
Structure Dynamic	Unsorted ok	Must remain sorted	Dispersed allocation



Performance Analysis

Access Complexity

- **Average Case:** $O(1)$ lookup operations, assuming an even, uniform distribution of key entries.
- **Worst Case:** $O(n)$ access operations if all inserted keys collide into the exact same index link sequence.

Search Strategies Trade-offs

Metric Type	Sequential	Binary Search	Hashing
Average Search Time	$O(n)$	$O(\log n)$	$O(1)$
Worst Search Time	$O(n)$	$O(\log n)$	$O(n)$
Memory Overhead	Minimum	Balanced	Table Space (α)
Structure Dynamic	Unsorted ok	Must remain sorted	Dispersed allocation

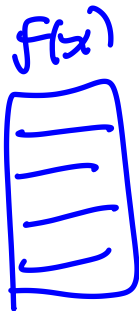
$O(n \log n)$ (merge sort)
 $n \log n + \log n$
 ← sort algorithm



Performance Analysis

Design Focus

To keep hashing performant, monitor table density profiles. Rehash the entire table into a larger footprint when $\alpha > 0.75$.



$\rightarrow$ diff

① $f(g(x))$
② new $h(x)$



Outline

$\mu \rightarrow 0x$

	0	1	2	3
0				
1			5	
2				
3		8		

$x = 5;$

$x \rightarrow 0x12$

$y = 7;$

$y = 8;$

$y \rightarrow 0x31$

pointer

memory allocation
 $\text{int } *z = \text{malloc}()$
 $*z = 0x33$



Thanks!

Questions?



Repo: <https://github.com/EngAndres/ud-public/tree/main/courses/computer-sciences-ii>

